

Title: Smart Home Automation System using
Arduino UNO

Subtitle: Minor Project Presentation | Embedded Systems &
Microcontrollers

Simulation Platform: Autodesk Tinkercad

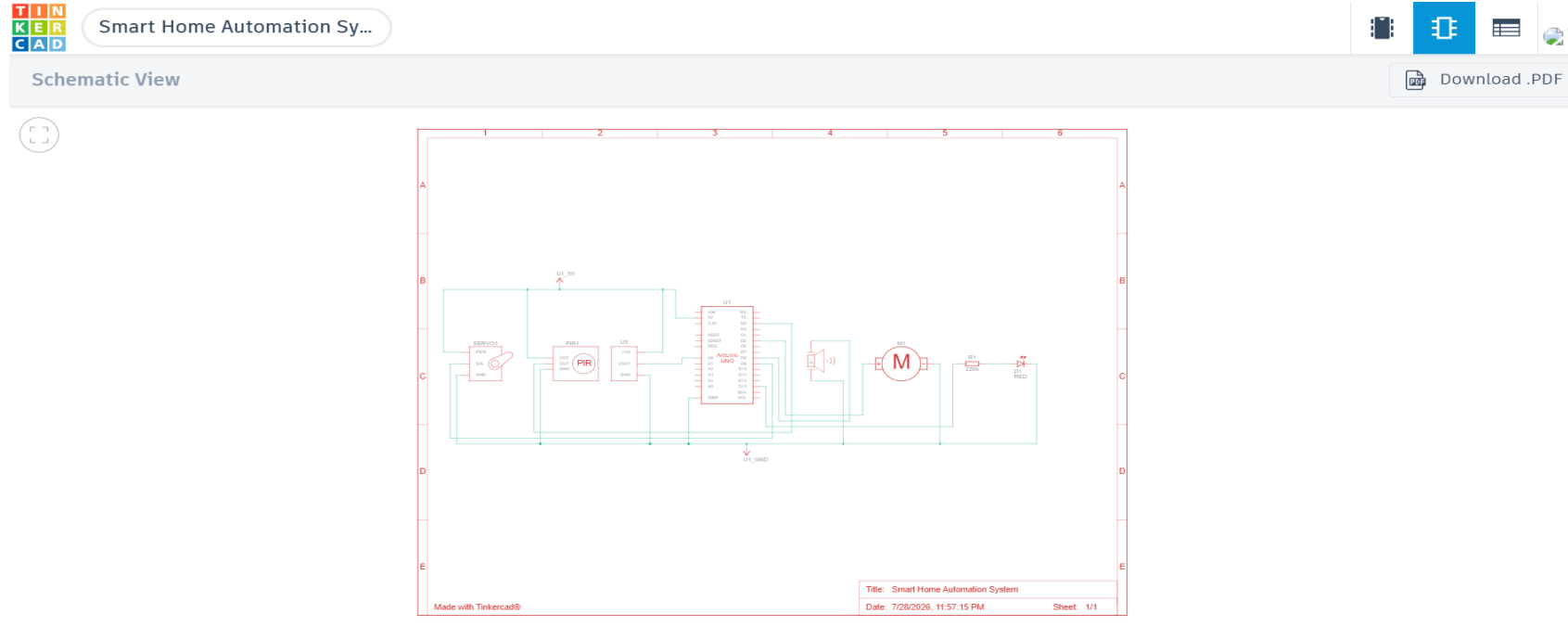
Presenter: Harshal Gangrade

Date: 29 July 2026

*The Problem:

- **Energy Waste:** Home appliances and lights stay ON in empty rooms.
- **Manual Effort:** Traditional switches are inconvenient for elderly or mobility-impaired individuals.
- **Safety Concerns:** Lack of real-time fire and temperature alerts in standard homes.
- **Energy Waste:** Home appliances and lights stay ON in empty rooms.
- **Manual Effort:** Traditional switches are inconvenient for elderly or mobility-impaired individuals.
- **Safety Concerns:** Lack of real-time fire and temperature alerts in standard homes.
- **Project Objectives:**
 - Automate lighting based on occupancy detection.
 - Control door access automatically using a servo motor.
 - Implement automated climate control (fan speed regulation via temperature).
 - Simulate and validate system performance in Tinkercad.

System Architecture & Circuit Workflow



Workflow Summary:

- **Inputs:** PIR Motion Sensor (Digital) & TMP36 Temperature Sensor (Analog).
- **Processing:** Arduino UNO processes sensor data in a continuous loop.
- **Outputs:** LED for lighting, Servo for door control, and DC Motor (PWM) for fan speed control.
- **Telemetry:** Live output streamed to Tinkercad Serial Monitor.

Hardware & Component Specifications

- **Component Function in System Pin Assignment Arduino UNO**

- Central microcontroller; processes input signals
- Board Brain

- **PIR Sensor**

- Detects human motion/presence
- Digital Pin 2

- **TMP36 Sensor**

- Reads ambient room temperature
- Analog Pin A0

- **Servo Motor**

- Simulates automated door lock (0degree to 90degree)
- Digital Pin 9

- **DC Motor**

- Simulates variable-speed cooling fan
- Digital Pin 5 (PWM)

- **LED / Buzzer**

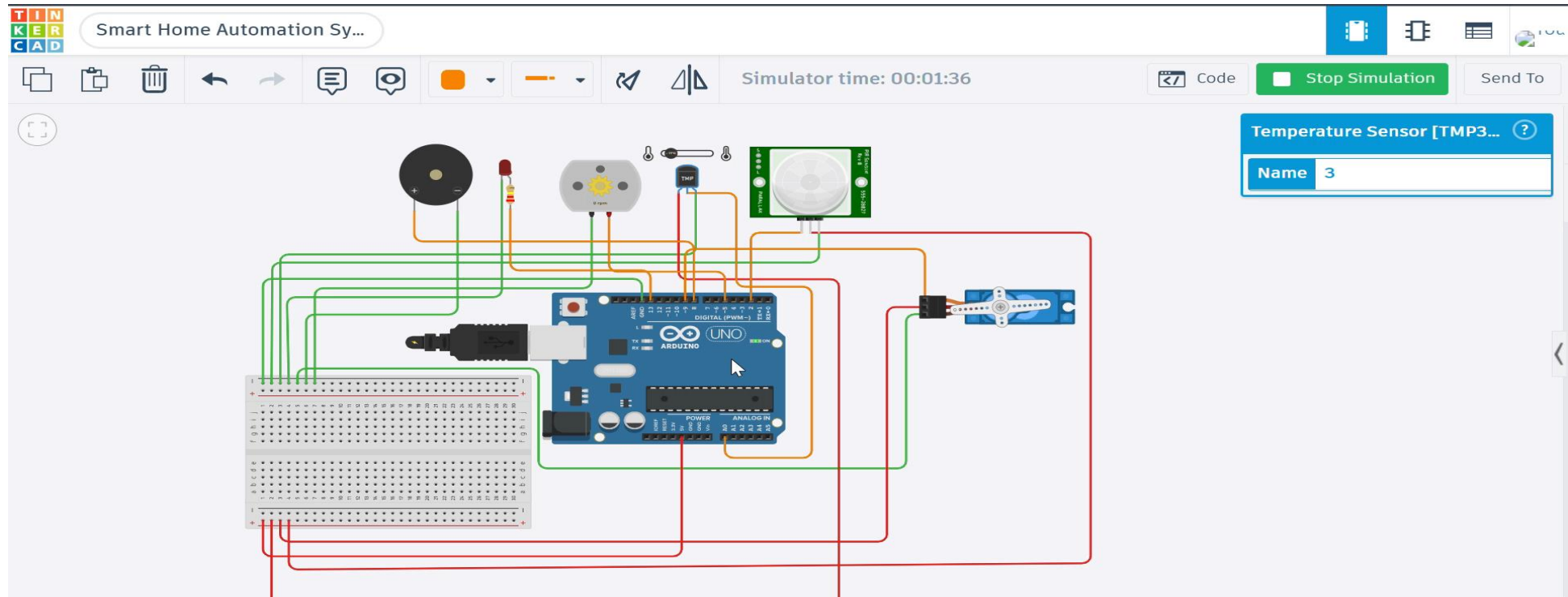
- Visual lighting indicator & alerts
- Digital Pin 13 / 8

Embedded C++ Logic

- **Motion & Door Control Logic:**
- If PIR Sensor reads HIGH, turn ON LED and set Servo to 90degree
- Else, turn OFF LED and reset Servo to 0° .
- **Temperature & Fan Control Logic:**
- Convert raw ADC steps to Celsius: $\text{Temp} = ((\text{ADC} \setminus 5.0 \setminus 1023.0) - 0.5) \times 100.0$
- If $\text{Temp} > 28^{\circ}\text{C}$, trigger DC Motor via PWM speed control.

Simulation Results & Verification

- **Motion Verification:** Moving the PIR test object instantly lights up Pin 13 LED and rotates the Servo to 90 degree
- **Temperature Verification:** Adjusting the TMP36 slider above 28degreeC activates the DC motor in real-time.
- **Serial Telemetry:** Live sensor readings and system states print every 500ms on the Serial Monitor.



Advantages & Future Scope

- **Key Advantages:**
- **Energy Efficiency:** Smart activation prevents power wastage.
- **Cost-Effective:** Simple, scalable components suitable for prototyping.
- **Safe Prototyping:** Tested virtually without risk of hardware damage.
- **Future Improvements:**
- **IoT Integration:** Add ESP8266/ESP32 for mobile app control (Blynk/Google Home).
- **Hazard Prevention:** Integrate MQ-2 smoke/gas sensor for fire safety.

Q&A / Technical Defense

- **Q1: How is raw ADC data converted to Celsius?**
- *Ans:* Arduino converts 0-5V into 0-1023 digital steps .The TMP36 outputs 10mV/C with a 500mV offset at 0degree C. Formula: $\text{Temp} = ((\text{ADC} * 5.0 / 1023.0) - 0.5) * 100$
- **Q2: Why use PWM for the DC fan motor instead of simple digital ON/OFF?**
- *Ans:* PWM controls average voltage by adjusting duty cycles. This enables precise speed regulation matching room temperature, rather than running strictly at full speed or stop.